

Hashing for Data Integrity and Non-repudiation

ISC2 CCSP: Certified Cloud Security Professional | Domain 2: Cloud Data Security

1. Integrity and Secrecy: Two Different Problems

Encryption and hashing are both tools that transform data, and both appear constantly in security discussions. The trouble is that they solve fundamentally different problems, and conflating them leads to real design failures. Encryption provides confidentiality: it hides the content of data from anyone without the correct key. Hashing provides integrity: it detects whether data has been changed. Neither can do the other's job.

A concrete illustration: you can encrypt a backup file to keep its contents secret, but if you want to know whether the backup is intact — whether it arrived uncorrupted after crossing a network or sitting in storage for three years — encryption alone cannot tell you. You need a hash. Conversely, hashing a file tells you nothing about who can read it. The hash value is public by design. Understanding which tool solves which problem is one of the foundational distinctions the CCSP exam tests repeatedly.

Key Point: Encryption = confidentiality (reversible with a key). Hashing = integrity (one-way, no key). Neither substitutes for the other. This distinction drives the entire design of cloud data security controls.

2. How Hash Functions Work

A hash function accepts an input of any length and produces a fixed-length output called a digest or hash value. The digest length is determined by the algorithm, not the input. SHA-256 always produces 256 bits. SHA-512 always produces 512 bits. Every possible input — a single character or a 10 GB database export — produces a digest of exactly the same length.

Three properties define a cryptographically secure hash function:

- **Deterministic:** The same input always produces the same output. Feed the identical file through SHA-256 a million times and you get the same result every time.
- **Avalanche effect:** Change a single bit anywhere in the input and the entire digest changes completely and unpredictably. There is no gradual shift.
- **Collision resistance:** It must be computationally infeasible to find two different inputs that produce the same digest. A hash function that can be deliberately collided loses its integrity guarantee entirely.
- **Pre-image resistance (one-way):** Given a digest, it must be computationally infeasible to find the original input. There is no key, no algorithm, no inverse operation.

3. Integrity Verification in Practice

The operational workflow is straightforward. Before storing a file — in cloud object storage, a database, an archive — compute its hash and record the digest. When you retrieve the file later, compute the hash

again and compare. If they match, the file is byte-for-byte identical to what was stored. If they differ, something changed: corruption during transit, silent bit-rot in storage, or deliberate tampering.

Use Case	How Hashing Is Applied
Backup integrity	Hash recorded at backup time; verified at each restore test to detect silent corruption
File transfer verification	Source computes hash before upload; destination recomputes after download and compares
Software distribution	Vendor publishes hash of installer; downloader verifies before executing
Audit trail entries	Each log entry hashed to detect tampering; chained hashes create tamper-evident logs
Container image verification	Docker and Kubernetes verify layer digests before running images

The hash does not tell you what changed, when it changed, or who changed it. It tells you one thing: the data you are reading is not identical to the data that existed when the original hash was taken.

4. SHA-2: The Current Standard

The Secure Hash Algorithm 2 family, developed by the NSA and published by NIST, is the current standard for cryptographic hashing in cloud security. SHA-256 and SHA-512 are the two most widely deployed members.

Algorithm	Digest Length	Common Uses	Status
SHA-256	256 bits	TLS certificates, code signing, HMAC in AWS APIs, Bitcoin	Secure
SHA-512	512 bits	High-security verification, document signing	Secure
SHA-384	384 bits	TLS 1.3 cipher suites, government applications	Secure
SHA-1	160 bits	Legacy systems, Git (migrating away)	Broken — 2017
MD5	128 bits	Non-security deduplication only	Broken — 2004

5. Broken Algorithms: SHA-1 and MD5

Both SHA-1 and MD5 have been cryptographically broken for integrity and security purposes. MD5 collision attacks were demonstrated in 2004 by researcher Xiaoyun Wang. Real-world damage followed: forged SSL certificates were created using MD5 collision vulnerabilities in 2008, allowing attackers to create fraudulent certificates trusted by major browsers.

Real-World Incident — SHattered: SHA-1 Collision (2017): Researchers from Google and CWI Amsterdam produced the first practical SHA-1 collision, creating two different PDF documents with identical SHA-1

digests. The attack cost roughly \$110,000 in cloud compute. An attacker who can create a collision can substitute one signed document for another without detection. After SHAttered, Certificate Authorities and browser vendors accelerated SHA-1 deprecation timelines.

Neither SHA-1 nor MD5 should be used in any new deployment where the hash is serving as a security control. They may appear legitimately in non-security contexts — for example, MD5 for file deduplication indexes where collision resistance is irrelevant. The critical distinction is context: the same algorithm can be appropriate for one use and completely inappropriate for another.

6. Git and Algorithm Agility

Git uses cryptographic hashes to identify every object in a repository. The hash of a commit uniquely identifies the entire repository state at that moment. For most of Git's history since 2005, those hashes were SHA-1. After SHAttered, the Git ecosystem faced a real engineering problem and began migrating to SHA-256 object IDs — a significant undertaking requiring updates across every tool, host, and client in the ecosystem.

This migration illustrates algorithm agility: the ability of a system to replace its underlying cryptographic algorithm without being rebuilt from scratch. A system that hardcodes a specific hash algorithm and cannot change it becomes a liability when that algorithm is broken. Algorithm agility is a design requirement for any long-lived cloud system that relies on cryptographic primitives.

7. Non-repudiation: Beyond Integrity

Integrity verification answers: is this data unchanged? Non-repudiation answers a different question: can the party who created or sent this data credibly deny having done so? In legal, audit, and compliance contexts, non-repudiation matters for contracts, financial transactions, and log entries that must withstand legal scrutiny.

A plain hash does not provide non-repudiation. You can prove the file has not changed since the digest was created. But you cannot prove who created the file, who possessed it, or who sent it. Anyone who has access to the hashing algorithm — which is everyone — can produce the same digest from the same file. The hash identifies the data; it does not authenticate the actor.

8. Digital Signatures: Integrity Plus Authorship

Digital signatures combine hashing with asymmetric encryption to deliver both integrity and non-repudiation:

- **Signing:** The signer hashes the document to produce a digest, then encrypts that digest with their private key. The encrypted digest is the digital signature, and it travels alongside the document.
- **Verification:** The recipient decrypts the signature using the signer's public key (which anyone can hold), independently hashes the document, and compares the two digests.

If the digests match, two things are simultaneously proven: the document has not changed since signing (integrity), and the signature was created by the holder of the private key (non-repudiation). This is the

mechanism behind code signing, document authentication, TLS certificate verification, and email security standards such as S/MIME and DKIM.

Security Property	Plain Hash	Symmetric Encryption	Digital Signature
Confidentiality	No	Yes	No (sign separately)
Integrity	Yes	Not by design	Yes
Non-repudiation	No	No (shared key)	Yes
Origin authentication	No	Partial (shared secret)	Yes

9. HMAC: Keyed Integrity for Cloud API Authentication

Hash-based Message Authentication Code (HMAC) incorporates a secret key into the hash computation, turning a plain hash into both an authentication and integrity control. Without the shared key, an attacker cannot produce a valid HMAC — even if they know the algorithm and can see the full request.

AWS Signature Version 4, which authenticates every AWS API request, uses HMAC-SHA256. The process:

- The application constructs a canonical representation of the request (method, URL, headers, body hash).
- It computes HMAC-SHA256 over that canonical string using its AWS secret access key.
- The resulting signature goes into the request's Authorization header.
- AWS recomputes the HMAC on its side and compares. A match confirms the request came from a caller holding the correct key and was not modified in transit.

Key Point: HMAC differs from plain hashing in one critical way: it requires a shared secret. An attacker who intercepts an HMAC-signed request cannot forge a valid request for a different action, because they do not have the key. HMAC provides integrity and authentication — plain SHA-256 provides neither on its own.

10. The One-Way Property: Why Passwords Use Hashing

Hashing is irreversible. Given a SHA-256 digest, there is no algorithm, no key, and no mathematical operation that recovers the original input. This makes hashing the right tool for password storage: a database that stores password hashes rather than plaintext cannot leak the original passwords even if the entire database is stolen.

However, SHA-256 is designed for speed — which is ideal for file integrity but dangerous for passwords. A GPU can test billions of SHA-256 guesses per second against stolen hashes. Password storage therefore requires purpose-built slow algorithms:

- **bcrypt:** Configurable work factor; slows each hash computation to a defined time. As hardware improves, the work factor is increased.
- **scrypt / Argon2:** Memory-hard algorithms that resist GPU-based attacks by requiring large amounts of memory per computation.
- **Salting:** A random per-user value appended to the password before hashing. Defeats rainbow table attacks by forcing independent computation per user.

11. Hashing in Cloud Data Security — Choosing the Right Tool

Requirement	Right Tool	Why
Verify a backup arrived intact	SHA-256 of the file	Fast, deterministic digest for comparison
Detect silent storage corruption	Periodic hash verification vs. stored digest	Catches bit-rot that does not trigger storage errors
Prove a log entry has not been altered	Hash-chaining or digital signature over entry	Hash alone verifies; signature adds authorship
Authenticate an API request	HMAC-SHA256 with shared secret key	Proves caller holds the key; request unmodified
Store a user password	bcrypt / Argon2 with salt	SHA-2 is too fast; slow algorithms resist brute-force
Sign code for non-repudiation	Digital signature (hash + private key)	Proves both integrity and authorship

12. Key Terms Glossary

Term	Definition
Hash function	Algorithm that maps any-length input to a fixed-length digest; deterministic and one-way
Digest / Hash value	The fixed-length output of a hash function — a data fingerprint
SHA-256 / SHA-512	Current standard hash algorithms (SHA-2 family); secure against known attacks
Collision	Two different inputs producing the same digest; breaks the integrity guarantee
Collision resistance	Property ensuring no two different inputs can be engineered to produce the same digest
Pre-image resistance	Property ensuring a digest cannot be reversed to recover the original input
Avalanche effect	Tiny input change causes completely different and unpredictable digest output
Non-repudiation	Inability of a party to credibly deny having performed an action
Digital signature	Hash of a document encrypted with the signer's private key; proves integrity and authorship
HMAC	Hash-based Message Authentication Code; incorporates a shared secret key into the hash
Algorithm agility	Architectural design allowing cryptographic algorithms to be swapped without full system replacement
Salt	Random per-user value added to a password before hashing to defeat rainbow table attacks

13. Quick Revision Checklist

- Hashing = integrity; encryption = confidentiality. Neither does the other's job.
- Four properties: deterministic, avalanche effect, collision resistance, pre-image resistance (one-way).
- SHA-256 and SHA-512 (SHA-2 family) are current standards and computationally secure.
- SHA-1 is broken — SHAttered (2017) demonstrated a practical collision for ~\$110,000.
- MD5 is broken — collision attacks since 2004; forged SSL certificates followed in 2008.
- Git migrated from SHA-1 to SHA-256 — a real-world algorithm agility case study.
- Digital signatures = hash + asymmetric encryption → integrity AND non-repudiation.
- Plain hashing provides integrity but NOT non-repudiation and NOT authentication of origin.
- HMAC adds a shared secret key → integrity AND authentication. AWS SigV4 uses HMAC-SHA256.
- Password storage needs slow algorithms (bcrypt, Argon2) with salts — SHA-2 alone is too fast.
- Algorithm agility is an architectural requirement for any long-lived cryptographic system.
- Key hashing applications: backup verification, tamper-evident logs, API auth (HMAC), password storage, code signing.

20 Exam-Based MCQs — Hashing and Non-repudiation

Test yourself after reading. These questions are written in real exam style — scenario-heavy, with plausible distractors. Read every explanation, including for questions you answered correctly.

Q1. Which property of a cryptographic hash function ensures that altering a single byte of input produces a completely different digest?

- A) Pre-image resistance
- B) Collision resistance
- C) Avalanche effect
- D) Algorithm agility

Answer: C **Explanation:** C is correct. The avalanche effect describes the property where a tiny change in input causes a completely unpredictable and wholesale change in the digest. A) Pre-image resistance means you cannot reverse the digest to find the original input — a different property about one-way computation. B) Collision resistance means no two different inputs can be engineered to produce the same digest — again a different property. D) Algorithm agility is not a hash function property but an architectural concept about replaceability.

Q2. An organisation stores the SHA-256 digest of each database backup at creation time. Three years later, the digest computed from the retrieved backup does not match the stored digest. What does this MOST likely indicate?

- A) The backup was encrypted in transit
- B) The hash algorithm is not collision-resistant
- C) The retrieved file differs from what was originally backed up
- D) SHA-256 has been deprecated

Answer: C **Explanation:** C is correct. A digest mismatch means the file content differs from when the original hash was taken — due to corruption, storage error, or tampering. A) Encryption in transit is irrelevant to hash comparison and would not cause a mismatch in stored vs. computed digests. B) A collision would produce a matching digest for different data — the opposite of a mismatch. D) SHA-256 is not deprecated; it remains current and secure.

Q3. Scenario: A security architect is designing API authentication for a cloud-native microservices platform. Services must prove to each other that requests are genuine and unmodified in transit. The services share a pre-provisioned secret key. Which mechanism *BEST* meets these requirements? Which mechanism *BEST* meets these requirements?

- A) SHA-512 applied to each request body
- B) HMAC-SHA256 using the shared secret key
- C) RSA digital signatures using each service's private key
- D) TLS encryption of the request payload

Answer: B **Explanation:** B is correct. HMAC-SHA256 incorporates the shared secret key into the hash computation, providing both integrity (request unaltered) and authentication (only a holder of the secret key can produce a valid HMAC). A) Plain SHA-512 has no key; any party can produce the same hash for the same data, so it does not authenticate origin. C) RSA digital signatures work but require asymmetric key infrastructure; the scenario specifies a shared secret key. D) TLS encrypts the channel but does not authenticate message content at the application layer.

Q4. Which statement *BEST* describes the difference between a digital signature and a plain hash of the same document?

- A) A digital signature is reversible; a hash is not
- B) A digital signature adds non-repudiation by encrypting the hash with the signer's private key
- C) A digital signature uses a symmetric key while a hash uses no key
- D) A hash provides non-repudiation while a digital signature provides only integrity

Answer: B **Explanation:** B is correct. A digital signature hashes the document then encrypts that digest with the signer's private key, binding authorship to the document — only the holder of the private key could produce a signature that decrypts correctly with the corresponding public key. A) Hashing is one-way; the digital signature does not make it reversible. C) Digital signatures use asymmetric (not symmetric) key pairs. D) The roles are reversed; digital signatures provide non-repudiation while plain hashes provide only integrity.

Q5. A development team continues using SHA-1 hashes to verify internally generated software packages, arguing that SHA-1 is adequate for low-risk internal tools. What is the *MOST* accurate response?

- A) SHA-1 is still acceptable internally as long as external systems use SHA-256

- B) SHA-1 must not be used for any integrity security control because practical collisions have been demonstrated
- C) SHA-1 is acceptable as long as the digests themselves are stored encrypted
- D) SHA-1 is only broken for certificate signing; it remains secure for file integrity

Answer: B **Explanation:** B is correct. SHAttered (2017) demonstrated a practical SHA-1 collision for roughly \$110,000 in cloud compute. Any system using SHA-1 as an integrity control is now vulnerable to an attacker substituting a malicious package with an identical SHA-1 hash. A) The internal/external distinction is irrelevant; a broken algorithm is broken wherever integrity is a security requirement. C) Encrypting the digest does not fix collision vulnerability. D) SHA-1 is broken for any collision-resistance application, not only certificate signing.

Q6. Scenario: A cloud security team reviews credential storage for a user-facing web application and finds that user passwords are stored as SHA-256 hashes with no additional processing. The security lead flags this as a significant vulnerability. What is the PRIMARY concern?

- A) SHA-256 is broken and vulnerable to collision attacks
- B) SHA-256 is too fast, making brute-force and dictionary attacks against the hashes computationally cheap
- C) SHA-256 hashes are reversible with sufficient computing power
- D) SHA-256 does not support salting and is therefore vulnerable to rainbow table attacks

Answer: B **Explanation:** B is correct. SHA-256 is designed for speed, which is appropriate for file integrity but dangerous for password storage. A GPU can test billions of SHA-256 password guesses per second against stolen hashes. Password storage requires intentionally slow algorithms such as bcrypt or Argon2. A) SHA-256 has not been broken; no collision attacks exist against it. C) SHA-256 is pre-image resistant and not reversible. D) SHA-256 supports salting; the issue is speed, not inability to salt.

Q7. What is HMAC's primary advantage over a plain cryptographic hash for verifying API requests?

- A) HMAC uses asymmetric keys, providing stronger non-repudiation
- B) HMAC is significantly faster to compute than SHA-256
- C) HMAC incorporates a shared secret, so only parties holding the secret can produce or verify a valid code
- D) HMAC produces a longer digest, reducing collision probability

Answer: C **Explanation:** C is correct. The key differentiator is the shared secret. A plain hash of a request can be reproduced by anyone who sees the request; HMAC cannot, because it requires the secret key. A) HMAC uses a symmetric shared secret, not asymmetric keys; it does not provide non-repudiation. B) HMAC performance is comparable to the underlying hash algorithm; speed is not its advantage. D) HMAC digest length matches the underlying hash (e.g., HMAC-SHA256 = 256 bits); it does not increase digest length.

Q8. Scenario: A vendor distributes software installers with accompanying SHA-256 checksums published on their website. A user downloads the installer and its checksum from a compromised mirror site. The

downloaded checksum matches the downloaded installer — but both the installer and checksum have been replaced by the attacker. What integrity limitation does this scenario demonstrate?

- A) SHA-256 is vulnerable to length-extension attacks
- B) Hash verification only works relative to the digest provided — if the attacker controls both the file and the digest, verification provides no protection
- C) Hash verification always requires HMAC to be effective
- D) SHA-256 has been broken and is no longer collision-resistant

Answer: B **Explanation:** B is correct. Hash verification compares a file against a provided digest. If an attacker substitutes both the file and the digest, verification passes — the user is comparing the malicious file against the attacker's digest, which matches by design. The fix is to obtain the digest from a separately trusted source. A) Length-extension attacks affect certain MAC constructions, not this scenario. C) HMAC does not help when the attacker controls the digest source entirely. D) SHA-256 has not been broken; this is an operational trust problem.

Q9. An organisation needs audit logs that remain tamper-evident for 15 years. Which approach provides the STRONGEST guarantee?

- A) Store SHA-256 digests of each log entry in the same table as the entries
- B) Hash-chain the logs so each entry includes the hash of the previous entry, with periodic digital signatures at checkpoints
- C) Encrypt all log entries with AES-256
- D) Store logs in object storage with versioning enabled

Answer: B **Explanation:** B is correct. A hash chain makes every entry dependent on all previous ones: altering or deleting any entry breaks the chain from that point forward. Periodic digital signatures create trusted checkpoints. A) Storing digests in the same table allows an attacker to replace both entry and digest. C) Encryption provides confidentiality; it does not prevent an authorised party from modifying entries. D) Versioning detects overwrites but does not cryptographically detect content tampering.

Q10. Which statement about Git's transition from SHA-1 to SHA-256 is MOST accurate from a cloud security perspective?

- A) The transition is unnecessary because SHAttered only affects certificate authorities
- B) The transition demonstrates algorithm agility in practice: a widely deployed system replacing a broken cryptographic primitive
- C) Git objects do not use hashing for security, so the transition has no security relevance
- D) The transition shows SHA-256 is also vulnerable and should be replaced with SHA-3

Answer: B **Explanation:** B is correct. Git's SHA-1 to SHA-256 migration is a real-world example of algorithm agility: a large distributed system updating its cryptographic foundation without replacement. A) SHAttered affected any system relying on SHA-1 collision resistance, including Git commit integrity. C) Git uses SHA-1 hashes as object identifiers with the assumption that no two commits share a hash; SHAttered broke that assumption. D) The migration is motivated by SHA-1's weakness; there is no implication that SHA-256 is similarly broken.

Q11. Scenario: A legal team reviewing electronic contracts needs to confirm the signing mechanism provides non-repudiation — meaning a signer cannot later claim they did not sign or that the document was altered after signing. Which technical mechanism provides BOTH integrity and non-repudiation?

- A) SHA-256 hash of each signed document stored in a database
- B) AES-256 encryption of each document after signing
- C) Digital signatures using each signer's RSA private key applied to the document hash
- D) HMAC-SHA256 with a shared key between the platform and the signer

Answer: C **Explanation:** C is correct. A digital signature (hash encrypted with the signer's private key) provides: integrity — any alteration breaks hash verification; non-repudiation — only the holder of the private key could have produced the signature. A) A plain hash provides integrity but not non-repudiation — anyone could have computed the hash. B) AES encryption provides confidentiality, not integrity or non-repudiation. D) HMAC with a shared key cannot provide non-repudiation because both parties hold the key and either could have generated it.

Q12. A cloud operations team hashes daily database exports with SHA-256 and stores digests in a write-protected log. What security property does this MOST directly provide?

- A) Confidentiality of database contents
- B) Non-repudiation of who performed the export
- C) Integrity verification — ability to detect whether the export has been altered since backup
- D) Authentication of the export process

Answer: C **Explanation:** C is correct. Storing a hash separately enables later comparison: if the recomputed hash differs from the stored digest, the export has been altered. A) SHA-256 does not encrypt data; it has no effect on confidentiality. B) A hash does not capture who performed the export; that requires audit logging and digital signatures. D) Hashing does not authenticate the process or the actor.

Q13. In the context of password storage, why is bcrypt preferred over SHA-256?

- A) bcrypt produces a longer digest than SHA-256
- B) SHA-256 cannot be reversed but bcrypt can, enabling password recovery
- C) bcrypt is intentionally slow with a configurable work factor, making brute-force attacks computationally expensive
- D) bcrypt incorporates asymmetric keys and provides non-repudiation

Answer: C **Explanation:** C is correct. bcrypt's configurable work factor slows each hash computation to a defined time (e.g., 100ms). A GPU that can test billions of SHA-256 passwords per second might only test thousands of bcrypt passwords per second. As hardware improves, the work factor is increased. A) Digest length is irrelevant to password storage security. B) Both SHA-256 and bcrypt are pre-image resistant and non-reversible. D) bcrypt is a symmetric key-free algorithm with no relation to asymmetric keys or non-repudiation.

Q14. Scenario: A cloud architecture team is designing a data pipeline that transfers large files between object storage buckets across regions. The receiving system must confirm each file arrived without corruption or tampering before processing. The solution should minimise additional infrastructure. What is the MOST appropriate integrity mechanism?

- A) Encrypt all files with AES-256 before transfer
- B) Compute SHA-256 of each file before upload; recompute and compare after download before processing
- C) Use HMAC-SHA256 with a shared key for each file
- D) Apply TLS 1.3 to all transfer connections

Answer: B **Explanation:** B is correct. Computing a hash before upload and verifying after download is a simple, infrastructure-light integrity check detecting corruption or tampering. A) AES-256 encryption provides confidentiality, not integrity verification. C) HMAC adds key management overhead appropriate for API authentication, not file transfer verification. D) TLS 1.3 protects data in transit but does not verify stored file integrity after transfer completes.

Q15. Which characteristic makes SHA-1 and MD5 unsuitable for security-critical integrity controls?

- A) Both are reversible with sufficient computational power
- B) Both have been shown vulnerable to practical collision attacks
- C) Both produce digests shorter than 128 bits, providing insufficient entropy
- D) Both require a secret key and therefore are not widely supported

Answer: B **Explanation:** B is correct. MD5 collision attacks were demonstrated in 2004; SHA-1 practical collision (SHAttered) was demonstrated in 2017. An attacker who can engineer collisions can substitute one file for another without detection. A) Neither SHA-1 nor MD5 is reversible; both are pre-image resistant. C) SHA-1 produces 160-bit digests and MD5 produces 128-bit digests; the issue is collision resistance, not digest length. D) Neither algorithm requires a key; they are keyless hash functions.

Q16. A security reviewer finds an application stores SHA-256 package digests in the same database as the packages themselves. What is the PRIMARY weakness?

- A) SHA-256 cannot be used to verify software packages
- B) Storing digests with the data they protect allows an attacker who compromises the database to replace both
- C) SHA-256 digests must be stored encrypted to be valid
- D) Package integrity requires digital signatures, not plain hashes

Answer: B **Explanation:** B is correct. Hash-based integrity verification depends on obtaining the digest from a source the attacker cannot simultaneously compromise. A compromised database allows replacement of both the file and its digest. The fix is separate, independently trusted digest storage. A) SHA-256 is appropriate for package verification. C) Encrypting digests does not address the trust anchor problem. D) Plain hashes are sufficient if the digest source is independently trusted; signatures are stronger but not strictly required.

Q17. Scenario: An e-commerce company's legal team requires the ability to prove in court that a specific customer approved a specific transaction on a specific date, and that the transaction record has not been altered since approval. Which mechanism BEST satisfies both the integrity and non-repudiation requirements?

- A) AES-256 encryption of each transaction record
- B) SHA-256 hash of each transaction record stored in a tamper-evident log
- C) Digital signatures applied to each transaction record using the customer's private key
- D) HMAC-SHA256 applied to each transaction using a shared platform key

Answer: C **Explanation:** C is correct. A digital signature with the customer's private key proves the record is unaltered (integrity via hash comparison) and that the customer approved it (non-repudiation via private key usage). A) Encryption provides confidentiality, not integrity or non-repudiation. B) A hash in a tamper-evident log proves integrity but not non-repudiation — it does not prove who created the record. D) HMAC with a shared platform key proves integrity but not non-repudiation; the platform holds the key and could generate it on the customer's behalf.

Q18. AWS Signature Version 4 uses HMAC-SHA256 to sign API requests. What does this mechanism specifically prevent?

- A) Interception of the request payload by a network attacker
- B) A man-in-the-middle attacker from modifying request parameters after signing
- C) The AWS service from logging the API call
- D) Replay attacks where the same signed request is sent multiple times

Answer: B **Explanation:** B is correct. The HMAC is computed over the canonical representation of the request. Any modification of parameters — URL, headers, body — produces a different HMAC that AWS's verification will reject. A) HMAC does not encrypt the request; TLS handles payload confidentiality. C) HMAC has no effect on logging; CloudTrail logs calls regardless of the signing mechanism. D) HMAC-SHA256 alone does not prevent replay attacks; that requires timestamps and nonces, which AWS SigV4 also uses separately.

Q19. Scenario: A build pipeline uses SHA-1 checksums to verify downloaded open-source dependencies before including them in production software. A security reviewer flags this as critical. The team wants to understand the specific attack scenario. What attack does SHA-1 verification of dependencies MOST directly enable?

- A) An attacker could decrypt the dependency checksums and recover source code
- B) An attacker could craft a malicious dependency with the same SHA-1 hash as the legitimate one, allowing it to pass verification
- C) SHA-1 hashes intercepted in transit could identify the exact dependency versions in use
- D) SHA-1 verification requires a shared secret key that could be stolen

Answer: B **Explanation:** B is correct. SHA-1 collision attacks allow crafting two different files with the same digest. A malicious package engineered to share the SHA-1 hash of the legitimate package would pass checksum verification and be included in the build — a supply-chain attack. A) SHA-1 hashes are one-way;

source code cannot be recovered from them. C) Intercepting a hash reveals nothing useful to an attacker; it is a fixed-length digest. D) SHA-1 requires no secret key; it is a keyless hash function.

Q20. Which of the following BEST describes the purpose of adding a random salt before hashing a password?

- A) It increases the length of the digest, making brute-force harder
- B) It makes the hash reversible so the original password can be recovered if needed
- C) It ensures that identical passwords produce different hashes, defeating precomputed rainbow table attacks
- D) It slows down the hash function, replacing the need for bcrypt or Argon2

Answer: C **Explanation:** C is correct. A salt is a random per-user value appended to the password before hashing. Two users with the same password produce different hashes because their salts differ. An attacker with a precomputed rainbow table (mapping common passwords to their hashes) cannot use it against salted hashes — they must recompute hashes independently for each user's salt. A) Salting does not change the digest length; the hash algorithm determines that. B) Salting does not make hashing reversible; the process remains one-way. D) Salting does not slow the hash function; that is bcrypt's or Argon2's work factor mechanism.